

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Proyecto No. 1

Análisis Semántico

Construcción — Semestre 2, 2026

Integrantes:

Camila Richter 23183

Marínés García 23391

Jose Antonio Mérida 201105

Catedrático: Santiago Solorzano

Sección: 20

Guatemala, 07 de Septiembre del 2026

Índice

Índice	2
1. Introducción	3
2. Antecedentes	3
3. Cuerpo del informe	4
3.1 Descripción general del lenguaje Compiscript	4
3.2 Herramientas y tecnologías utilizadas	4
3.3 Arquitectura general del compilador	4
3.4 Análisis léxico y sintáctico	5
3.5 Análisis semántico	5
3.5.1 Sistema de tipos	5
3.5.2 Tabla de símbolos y manejo de ámbitos	6
3.5.3 Reglas semánticas implementadas	6
3.6 Recuperación de errores	7
3.7 Interfaz gráfica (IDE)	7
3.8 Pruebas y validación	8
3.9 Hallazgos y correcciones durante la revisión final	8
4. Conclusiones	9
5. Referencias	9
6. Apéndices	9
Apéndice A — Enlaces del proyecto	9
Apéndice B — Ejemplo de programa Compiscript válido	10
Apéndice C — Ejemplo de salida con errores semánticos	10

1. Introducción

Este informe documenta el Proyecto 1 del curso Compiladores 2: la construcción de un analizador semántico completo para Compiscript, un lenguaje de programación de propósito general. El proyecto extiende la fase de análisis léxico y sintáctico construida previamente con un sistema de tipos, una tabla de símbolos y la validación de todas las reglas semánticas del enunciado, integrado en un IDE con interfaz gráfica que permite escribir, seleccionar y ejecutar programas Compiscript y visualizar los errores, el árbol de derivación y la tabla de símbolos. El documento describe la arquitectura de la solución, las decisiones de diseño y su justificación, las reglas semánticas implementadas, la recuperación de errores en las tres fases del análisis, y la evidencia de cumplimiento de las restricciones obligatorias: uso de una herramienta generadora, interfaz gráfica funcional y reporte de múltiples errores en una sola ejecución.

2. Antecedentes

El curso entrega el compilador por fases acumulativas, según la estructura clásica de Aho et al. (2006): análisis léxico, sintáctico y semántico, generación de código intermedio y de código final. Este informe corresponde a la fase semántica.

El punto de partida fue el laboratorio anterior del curso, en el que el mismo equipo construyó el analizador léxico y sintáctico de Compiscript con ANTLR4, junto con un IDE web mínimo. Se verificó que la gramática oficial para el análisis semántico fuera idéntica, carácter por carácter, a la ya implementada, por lo que se reutilizó íntegramente esa base (gramática, lexer y parser generados, CLI, servidor HTTP e IDE) en lugar de reconstruirla.

La única modificación previa fue agregar el tipo float: el enunciado exige verificar tipos en operaciones aritméticas entre integer y float, pero la gramática oficial solo contemplaba boolean, integer y string. Se extendió el lexer con un literal decimal (FloatLiteral), se agregó float como alternativa válida de BaseType y se regeneró el analizador con ANTLR, sin afectar las reglas léxicas ni sintácticas existentes.

3. Cuerpo del informe

3.1 Descripción general del lenguaje Compiscript

Compiscript es un lenguaje de propósito general cuya sintaxis reproduce varias construcciones de TypeScript (Microsoft, 2023): declaración de variables con `let`, `var` y `const`, anotaciones de tipo explícitas, funciones con parámetros y tipo de retorno anotados, y clases con miembros tipados. El lenguaje soporta:

- Tipos primitivos: `boolean`, `integer`, `float` (agregado por el equipo, ver sección 2), `string` y `null`.
- Arreglos, incluyendo arreglos multidimensionales (`integer[][]`) y acceso por índice.
- Funciones con parámetros opcionalmente tipados, recursión, funciones anidadas y closures.
- Clases con atributos y métodos, un método constructor especial, herencia simple (`class B: A`) y la referencia `this`.
- Control de flujo completo: `if/else`, `while`, `do-while`, `for`, `foreach`, `switch/case/default`, `break`, `continue`, `return`, `try/catch`.
- Expresiones aritméticas, lógicas, relacionales y de igualdad con la precedencia estándar, además de un operador ternario.

3.2 Herramientas y tecnologías utilizadas

El enunciado solo exige que el analizador léxico y sintáctico se genere con una herramienta generadora; el resto quedó a criterio del equipo:

- ANTLR4 4.13.2: genera el lexer, el parser y las clases base `Listener/Visitor` (Parr, 2013) desde `src/grammar/Compiscript.g4`; se usó el patrón `Visitor` para el recorrido semántico.
- Python 3.9 con `antlr4-python3-runtime` para el analizador semántico, por continuidad con la fase anterior.
- FastAPI y Uvicorn para exponer el analizador como servicio HTTP local.
- React 18 con TypeScript, Vite y Monaco Editor (el motor de Visual Studio Code) para el IDE web.
- `pytest` para la batería de pruebas automatizadas.

3.3 Arquitectura general del compilador

La arquitectura sigue el flujo clásico de las primeras fases de un compilador (Aho et al., 2006): tokenización, construcción del árbol de derivación y recorrido de ese árbol para verificar la semántica y construir la tabla de símbolos. En este proyecto:

- `src/grammar/Compiscript.g4`: fuente de verdad de la sintaxis; ANTLR genera desde ella el lexer, el parser y el visitor en `src/generated/`, código que nunca se edita a mano.
- `src/compiler.py`: orquesta el proceso; recolecta errores léxicos y sintácticos con un `ErrorListener` personalizado y, solo si no hubo errores, ejecuta el `SemanticChecker` sobre el árbol.

- `src/semantic/checker.py`: la clase `SemanticChecker`, el `Visitor` con un método por cada regla gramatical relevante.
- `src/semantic/types.py` y `src/semantic/symbols.py`: sistema de tipos y tabla de símbolos, independientes de las clases generadas por ANTLR y reutilizables en las fases de código intermedio y MIPS.
- `src/server.py`: API HTTP consumida por el IDE en frontend/, que muestra errores, árbol de derivación y tabla de símbolos.

Esta separación en capas, decidida antes de repartir el trabajo (ver 3.5.4), deja `checker.py` como único punto de integración entre las tres áreas del equipo.

3.4 Análisis léxico y sintáctico

El analizador léxico y sintáctico, generado íntegramente por ANTLR4, se heredó sin cambios estructurales del laboratorio previo (más la adición del tipo `float` descrita en la sección 2). El enunciado exige que el lexer continúe procesando la entrada tras un carácter no reconocido y que el parser se recupere de los errores; ANTLR provee ambas capacidades de forma nativa mediante su recuperación en modo pánico, y solo se personalizó el formato de los mensajes (en español, con línea y columna) con un `ErrorListener` propio (`src/error_listener.py`). La sección 3.6 muestra la evidencia empírica.

3.5 Análisis semántico

El análisis semántico verifica que un programa sintácticamente válido tenga sentido según las reglas del lenguaje: coherencia de tipos, nombres declarados y restricciones que la gramática libre de contexto no expresa (Aho et al., 2006). `SemanticChecker` extiende `CompiscriptVisitor` con un método por regla relevante; cada uno devuelve el `Type` de la subexpresión visitada, así los tipos se propagan desde las hojas del árbol hacia arriba.

3.5.1 Sistema de tipos

El sistema de tipos (`src/semantic/types.py`) define la jerarquía `Type`: `IntegerType`, `FloatType`, `BooleanType`, `StringType`, `NullType`, `VoidType`, `ArrayType`, `FunctionType` y `ClassType`, más dos tipos sin equivalente en el lenguaje fuente:

- `ErrorType`: se devuelve tras reportar un error, en lugar del tipo real; es compatible con cualquier tipo para que una subexpresión ya inválida no genere una cascada de errores derivados.
- `UnknownType`: variable declarada sin anotación ni valor inicial (`let x;`); acepta cualquier valor hasta la primera asignación, cuando su símbolo se actualiza (`narrowing`) al tipo real de forma permanente.

Cada tipo implementa `is_assignable_to`, que decide si un valor puede asignarse, pasarse como argumento o compararse. Las reglas acordadas: promoción de `integer` a `float` (nunca al revés), `null` asignable a cualquier arreglo o clase, y una clase asignable a sus superclases.

3.5.2 Tabla de símbolos y manejo de ámbitos

La tabla de símbolos (`src/semantic/symbols.py`) es un árbol permanente de objetos `Scope`, cada uno con su tipo (`GLOBAL`, `FUNCTION`, `CLASS` o `BLOCK`), su tabla local de símbolos, una referencia a su padre —usada en la resolución de nombres— y una lista de hijos, que lo hace navegable como árbol y no como simple pila. La pila de scopes abiertos (`SymbolTable`) es transitoria; el árbol permanece completo al terminar el análisis y es lo que muestra el visor del IDE y lo que reutilizarán las fases de TAC y MIPS.

La tabla de símbolos soporta las cuatro operaciones fundamentales sobre las que se evalúa el proyecto:

- Insertar: `Scope.declare(symbol)` agrega un símbolo y devuelve `False` si el nombre ya existía en ese scope; así se detecta la redeclaración sin bloquear el shadowing en scopes anidados.
- Recuperar: `Scope.resolve(nombre)` busca en el scope actual y sube por la cadena de padres hasta el global, lo que resuelve nombres en bloques y funciones anidadas y closures sin código especial.
- Actualizar: el campo `type` de un `Symbol` se reasigna en la primera asignación de una variable sin anotación (narrowing), como sentencia (`x = valor;`) o como expresión (`print(x = valor)`).
- Manejo de ámbitos: `SymbolTable.enter_scope/exit_scope` abren y cierran un ámbito al entrar a un bloque, función o cuerpo de clase, dentro de un `try/finally` para no dejar la pila desbalanceada ante un error.

3.5.3 Reglas semánticas implementadas

Se implementaron todas las categorías de reglas semánticas especificadas en el enunciado:

- Sistema de tipos: operadores aritméticos (+, -, *, /, %, con promoción `integer→float` y concatenación de strings), lógicos (&&, ||, !) y de comparación (==, !=, <, <=, >, >=); tipos en asignaciones; inicialización obligatoria de constantes, garantizada por la gramática.
- Manejo de ámbito: resolución de nombres en bloques anidados; error por uso de variables no declaradas; prohibición de redeclarar un identificador en el mismo ámbito (permitiendo shadowing en ámbitos anidados); un entorno de símbolos nuevo por cada función, clase y bloque.
- Funciones: número y tipo de argumentos (coincidencia posicional) y tipo de retorno; recursión y funciones anidadas con closures, que surgen de la cadena de scopes; redeclaración de funciones, pues no hay sobrecarga.
- Control de flujo: las condiciones de `if`, `while`, `do-while`, `for` y `switch` deben ser de tipo booleano; `break` y `continue` solo son válidos dentro de un bucle (con un contador de profundidad de anidamiento); `return` solo dentro de una función.
- Clases y objetos: existencia de atributos y métodos accedidos con “.”, en lectura y escritura (`obj.campo` y `obj.campo = valor`); aridad y tipos del constructor en `new`; `this` solo dentro de una clase; miembros heredados por la cadena de herencia.

- Listas y estructuras: verificación de que los elementos de un arreglo compartan un tipo compatible (con la misma promoción integer→float) y de que el índice sea de tipo integer, permitiendo indexar solo arreglos.
- Generales: detección de código muerto (instrucciones después de return, break o continue en el mismo bloque) y de expresiones sin sentido semántico, como multiplicar una función; validación de declaraciones duplicadas de variables y de parámetros.

3.6 Recuperación de errores

El enunciado prohíbe que el análisis se detenga en el primer error, en cualquiera de las tres fases, y exige evitar mensajes repetitivos o derivados. Ambas restricciones se verificaron de forma empírica con archivos de errores intencionales: un archivo de clases y arreglos con siete errores semánticos de categorías distintas (miembro inexistente, argumentos de constructor, clase no declarada, this fuera de una clase e indexación de arreglos) se reporta completo en una sola ejecución, y el Apéndice C muestra otra salida real del IDE con cinco errores independientes.

El mecanismo de recuperación es distinto en cada fase:

- Léxico: ANTLR continúa tokenizando después de un carácter no reconocido; el ErrorListener personalizado registra el error (en español, con línea y columna) sin detener el proceso.
- Sintáctico: recuperación en modo pánico incorporada en ANTLR, que descarta tokens hasta un punto de sincronización razonable y continúa el parseo; verificado con archivos de hasta tres errores sintácticos, todos reportados.
- Semántico: el SemanticChecker nunca lanza una excepción ni interrumpe el recorrido; cada regla que detecta un problema agrega un mensaje a una lista acumulada (SemanticErrorList) y continúa. Fue un principio de diseño desde la Fase 0.

Para evitar la cascada de errores derivados, toda subexpresión con un error ya reportado devuelve ErrorType (ver 3.5.1); al ser compatible con cualquier tipo en las comprobaciones de asignabilidad, el error no se redispara en cada nodo que la contiene. El código muerto reporta un único error por bloque inalcanzable, no uno por instrucción. Se verificó además que un archivo con una llave de apertura sin cierre se procesa en milisegundos, sin señales de ciclo infinito.

3.7 Interfaz gráfica (IDE)

El enunciado exige una interfaz gráfica funcional y estética, con selección del archivo de entrada desde ella y resultados mostrados dentro de la interfaz. Se construyó un IDE de navegador (frontend/, React + TypeScript + Vite) con:

- Explorador de archivos: panel lateral con el árbol del área de trabajo, desde el cual se abre, crea, renombra o elimina cualquier archivo .cps; es el mecanismo de selección exigido por el enunciado.

- Editor de código: Monaco Editor (el motor de Visual Studio Code) con resaltado de sintaxis propio para Compiscript (palabras clave, tipos, literales, comentarios).
- Panel de salida: muestra, tras presionar Run, cada error léxico, sintáctico o semántico encontrado, junto con un mensaje final de estado.
- Visor de árbol de derivación: pestaña que se abre automáticamente tras cada ejecución, con una representación visual y colapsable del árbol de parseo.
- Visor de tabla de símbolos: pestaña que se abre cuando el análisis semántico se ejecutó, con cada ámbito (global, función, clase, bloque) anidado en árbol y los símbolos declarados, su tipo y su ubicación.

Cada ejecución persiste su resultado en el área de trabajo (un `.out` con la salida y un `.tree` con el árbol en JSON), reabribles desde el explorador.

3.8 Pruebas y validación

El enunciado exige que cada grupo escriba sus propios archivos de prueba. El equipo construyó una batería de 49 archivos `.cps` propios, organizados en siete categorías correspondientes a las reglas de la sección 3.5.3, cada una con casos válidos (deben analizar sin errores) e inválidos (deben reportar al menos un error). La batería se ejecuta con `pytest` mediante `make test`.

La batería se reparte así, en formato válidos/inválidos: ámbito 4/6; sistema de tipos 4/6; funciones 2/4; control de flujo 3/5; clases 3/6; arreglos 1/3; y generales 0/2. Los responsables principales fueron Camila Richter (ámbito), Marínés García (tipos y funciones) y José Antonio Mérida (control de flujo, clases y arreglos), con la categoría general compartida.

La suite incluye además pruebas de regresión (`src/tests/test_smoke.py`) que corren cada muestra del área de trabajo por el pipeline completo, y pruebas específicas de la extensión del tipo `float`. En total ejecuta 62 pruebas: 61 pasan y 1 se omite de forma esperada (la categoría “generales” solo contiene casos inválidos por diseño).

3.9 Hallazgos y correcciones durante la revisión final

En la revisión integral previa a la entrega se verificó de forma empírica cada restricción del enunciado y se encontraron dos defectos reales, ambos corregidos y cubiertos con pruebas de regresión:

- Asignación a propiedades sin validar: la lectura de `obj.campo` validaba existencia y tipo, pero la escritura (`obj.campo = valor`; incluido `this.campo = valor`; en un constructor) no verificaba nada. Se corrigió reutilizando la lógica de resolución de miembros de la lectura, en forma de sentencia y de expresión.
- Tipo de clase desconectado en anotaciones explícitas: `let a: Animal = new Animal();` construía un `ClassType` nuevo y vacío en lugar de reutilizar la clase declarada, por lo que no se encontraban sus miembros. No se detectó antes porque las pruebas usaban

inferencia de tipo (`let a = new Animal();`). Se corrigió resolviendo el nombre de la clase contra la tabla de símbolos.

4. Conclusiones

1. Se construyó un analizador semántico completo para Compiscript sobre la base léxica y sintáctica del laboratorio anterior, sin reescribirla y extendiéndola únicamente con el tipo `float` que las reglas de tipos del enunciado requerían. El sistema de tipos —con promoción numérica, `ErrorType` para evitar cascadas y `UnknownType` para inferencia diferida— junto con una tabla de símbolos organizada como árbol permanente de ámbitos permitieron implementar la totalidad de las reglas semánticas especificadas: tipos, manejo de ámbito, funciones, control de flujo, clases y objetos, y listas.
2. Se verificó de forma empírica, no solo por diseño, el cumplimiento de las restricciones obligatorias del enunciado: recuperación de errores en las tres fases, ausencia de mensajes repetitivos o derivados, uso de una herramienta generadora (ANTLR4), e interfaz gráfica desde la que se selecciona el archivo de entrada y se visualizan los errores, el árbol de derivación y la tabla de símbolos. La revisión final encontró y corrigió dos defectos reales en la escritura de propiedades de objetos, lo que confirma la importancia de probar casos que combinan varias reglas semánticas a la vez.
3. Dividir el trabajo por categorías de reglas semánticas, y no por los componentes de la ponderación, permitió que las tres personas avanzaran en paralelo desde el inicio sobre la interfaz común acordada en la Fase 0. El sistema de tipos y la tabla de símbolos se diseñaron para reutilizarse sin cambios estructurales en las siguientes entregas del curso: generación de código intermedio (TAC) y de código ensamblador MIPS.

5. Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson Addison-Wesley.

Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.

Microsoft. (2023). *TypeScript Handbook*.
<https://www.typescriptlang.org/docs/handbook/intro.html>

6. Apéndices

Apéndice A — Enlaces del proyecto

- Repositorio: <https://github.com/Jose-Merida-UVG/Compilers-2-CC3032> (rama proyecto1).
- Demo en video de la fase léxica/sintáctica (laboratorio previo): <https://youtu.be/lowDshTJ0TI>.

- Documentación adicional: docs/plan-proyecto1.md (plan y división), docs/ModuloAmbito.md (tabla de símbolos y ámbito), docs/ModuloTiposFunciones.md (tipos y funciones).

Apéndice B — Ejemplo de programa Compiscript válido

Extracto de valido_propiedad_annotacion_explicita.cps: herencia, constructor, this, y acceso y asignación de propiedades.

```
class Animal {  
  var nombre: string;  
  function constructor(nombre: string) { this.nombre = nombre; }  
}  
let a: Animal = new Animal("Rex"); a.nombre = "Firulais"; print(a.nombre);
```

Figura 2. Ejemplo de programa Compiscript que analiza sin errores.

Apéndice C — Ejemplo de salida con errores semánticos

Salida real del analizador sobre un archivo con cinco errores de categorías distintas en una sola ejecución:

```
let a: integer = "no soy integer";  
let b: boolean = a + 5;  
print(c);  
function f(x: integer): integer { return "tampoco soy integer"; }  
f(1, 2, 3);
```

Salida:

Error semántico en línea 1: la variable 'a' se declaró como integer pero se inicializa con string.
Error semántico en línea 2: la variable 'b' se declaró como boolean pero se inicializa con integer.
Error semántico en línea 3: la variable 'c' no ha sido declarada.
Error semántico en línea 5: el valor de retorno debe ser de tipo integer; se encontró string.
Error semántico en línea 7: se esperaban 1 argumento(s) y se recibieron 3.
Se encontraron 5 errores durante el análisis.

Figura 3. Cinco errores semánticos independientes, reportados en una sola ejecución.